Ortak Hesap

Architecture and protocols · Milestone 1 · software design pattern, data flow and interfaces

Team 5 · MTM4681 Mobile Programming
Hakan İnce & İlhan Ünal
Flutter · Material 3 · Firebase

1 Chosen pattern: MVVM with a Repository layer

We will build Ortak Hesap in **Flutter** and organise the code with **MVVM** (Model–View–ViewModel) plus a **Repository** layer between the ViewModels and the data sources. Flutter UIs are declarative: a widget is a function of state. MVVM fits this naturally, because the *View* only renders state exposed by a *ViewModel* and forwards user events back to it. In classic MVC the controller would have to push updates into the view, which is awkward in Flutter and tends to mix widget code with business logic. Keeping the logic in ViewModels makes it testable without a device. Alternatives such as BLoC or Redux would also work, but they add more boilerplate than a two-person project needs; we use `provider` with `ChangeNotifier`, which is simple and taught in the course.

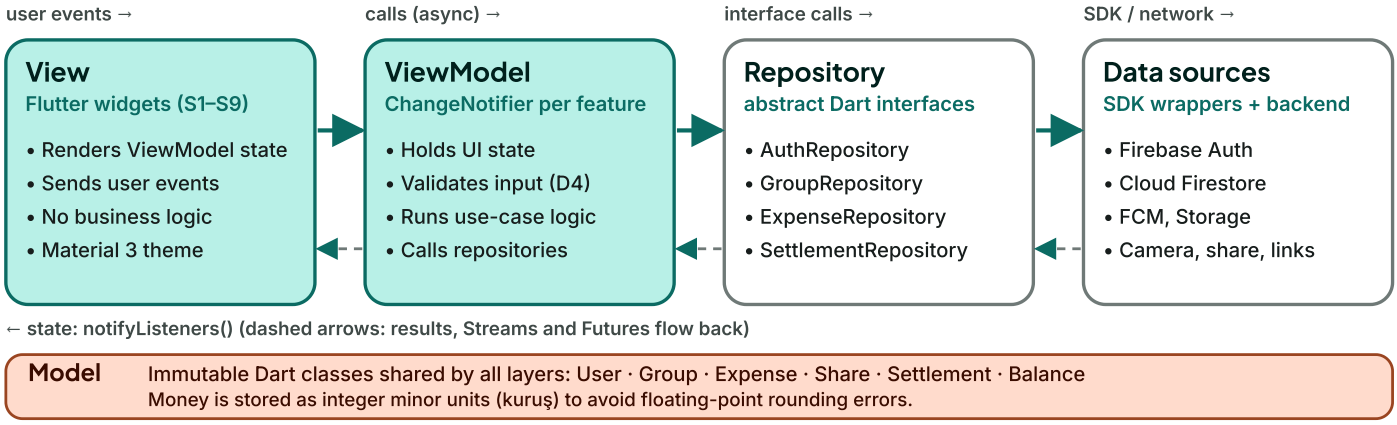


Figure 1. Layers of Ortak Hesap. Dependencies point from left to right; the View never talks to Firebase directly.

2 How the parts map to our screens

Screens (see wireframes)	ViewModel	Model objects used	Responsibility
S1a, S1b Onboarding, Sign in	`AuthViewModel`	User	Sign in / sign up, session state (decision D1 “signed in?”)
S2, S2b My Groups	`GroupsViewModel`	Group, Balance	Streams the user’s groups and balances (decision D2 “has a group?”)
S3, S4 Create / Join group	`CreateGroupVM`, `JoinGroupVM`	Group, invite code	Creates a group, builds invite link and QR, validates a code before joining
S5 Group detail	`GroupDetailViewModel`	Group, Expense, Balance	Expense list grouped by date, headline balance
S6, S6b Add expense	`AddExpenseViewModel`	Expense, Share	Form state, equal/custom split, validation (D3, D4), optional receipt photo
S7, S7b, S7c Balances	`BalancesViewModel`	Balance, Settlement	Suggested payments, “Mark as paid”, all-settled state (D5)
S8, S9 Activity, Profile	`ActivityVM`, `ProfileVM`	Activity, User	Feed of events; language, currency, theme, notifications

Planned folder structure (feature-first):

```
lib/
  app/        routes (go_router), theme
  core/       money.dart, debt_solver.dart
  data/       models/, repositories/ (interfaces
              + Firebase impl.), sources/
  features/   auth, groups, expenses, balances,
              activity, profile; each feature has
              view/ (screens, widgets) and
              viewmodel/ (ChangeNotifier)
```

Why this helps the team

- **Testable:** ViewModels depend on repository *interfaces*, so unit tests inject in-memory fakes (no network).
- **Replaceable backend:** only `data/` knows about Firebase.
- **Parallel work:** one of us builds views from the Figma mockups while the other builds ViewModels and repositories against the agreed interfaces.

3 Data model and storage

Data lives in **Cloud Firestore** (document database with real-time listeners and an offline cache). Each group is a document with two sub-collections, so a member's read access can be checked with one rule.

Collection	Main fields	Notes
`users/{uid}`	displayName, email, photoUrl, locale, currency, fcmTokens[]	Created at first sign-in
`groups/{gid}`	name, type (home | trip | other), memberIds[], inviteCode, createdBy, createdAt	Invite code is unique, 4–6 characters
`groups/{gid}/expenses/{eid}`	title, amountMinor, paidBy, splitType (equal | custom | percent), shares{uid: minor}, receiptUrl?, date	Sum of shares must equal amountMinor
`groups/{gid}/settlements/{sid}`	from, to, amountMinor, createdAt, createdBy	Written by "Mark as paid"
`invites/{code}`	groupId, expiresAt	Lookup for the Join screen (S4)

4 Key flow: adding an expense (S6 → S5)

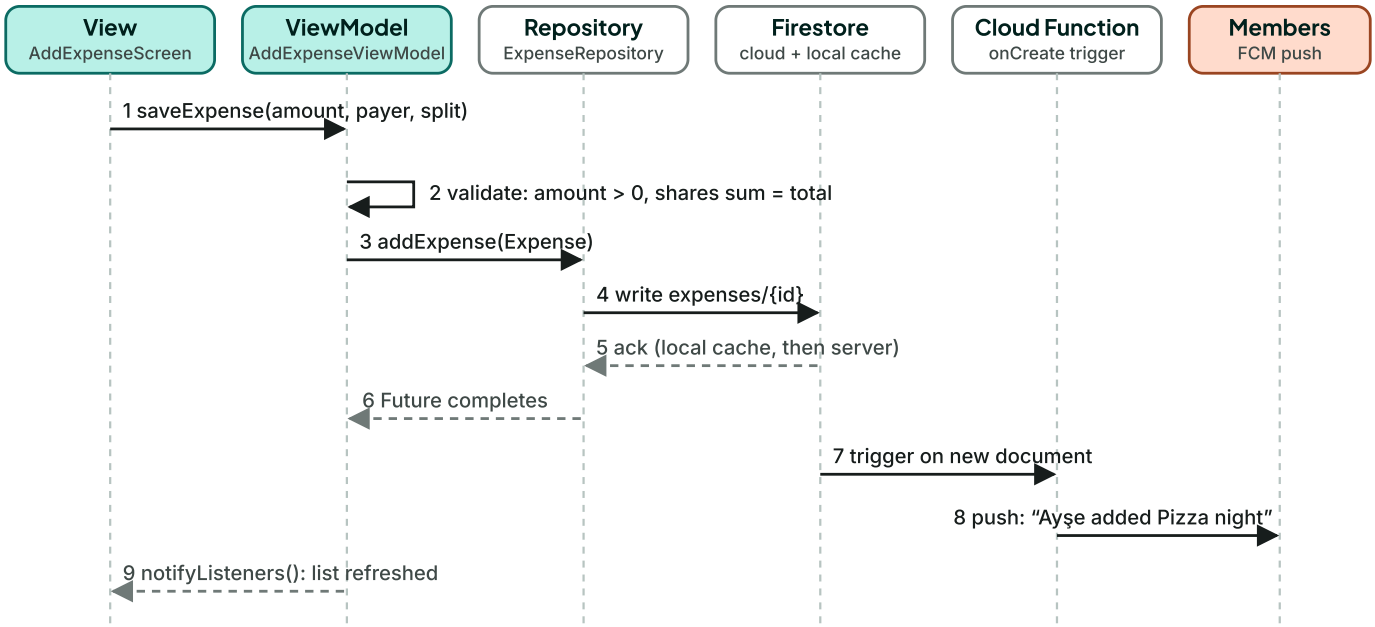


Figure 2. Sequence for "Save expense". The screen updates from the local cache immediately (offline-first); the push to other members happens afterwards.

5 Algorithms in the Model layer

Money and equal split. Amounts are integers in kuruş (₺86,90 = 8690). For *n* people, each share is `floor(total / n)` and the `total mod n` remaining kuruş go one by one to the first members. Example: 8690 / 4 → 2173, 2173, 2172, 2172, which always sums to the total (this matches the numbers in the mockups).

Balance of a member = what they paid − their shares + settlements received − settlements paid. Balances sum to zero within a group.

Simplifying debts (greedy, at most *n*−1 payments):

```
creditors = members with balance > 0
debtors   = members with balance < 0
while both lists are not empty:
    c = creditor with the largest balance
    d = debtor with the largest |balance|
    pay = min(c.balance, |d.balance|)
    emit "d pays c: pay"
    c.balance -= pay;  d.balance += pay
```

Flat 4B: You +120, Ayşe −80, Mert −40, Can 0 gives two payments (Ayşe→You 80, Mert→You 40), as shown on screen S7.

6 Protocols and interfaces

Protocol / interface	Purpose in Ortak Hesap	Screens	Package / platform API
HTTPS (TLS 1.2+)	Encrypts all traffic between the app and Firebase; no plain HTTP is used.	all	Firebase SDKs, Android network security config
OAuth 2.0 / OpenID Connect and e-mail + password	Sign in with Google or e-mail. The backend returns a signed ID token (JWT) that proves who the user is.	S1b	`firebase_auth`, `google_sign_in`
Cloud Firestore protocol (real-time listeners over a persistent connection)	Stores groups, expenses and settlements; streams changes so every member sees updates instantly; offline cache lets users add expenses without network.	S2, S5, S7, S8	`cloud_firestore`
Firebase Cloud Messaging (HTTP v1 API, Android push)	Sends push notifications: "Ayşe added Pizza night", "Mert paid you back". Needs the runtime notification permission on Android 13+.	S8, S9	`firebase_messaging`, Cloud Functions
Deep links (Android App Links, HTTPS URL `ortakhesap.app/join/K7P2`)	Opens the app straight to the Join screen from a shared invite link and pre-fills the code.	S4	`app_links`, `go_router`
Camera + QR decoding (runtime permission)	Scans a group QR code and takes a receipt photo; if permission is denied, the user can still type the code or save without a photo.	S4, S6	`mobile_scanner`, `image_picker`
Share sheet (Android `ACTION_SEND` intent)	Lets the user send the invite link through any messaging app.	S3	`share_plus`
Cloud Storage for Firebase (HTTPS, access-controlled URLs)	Holds receipt photos; only group members can read them.	S5, S6	`firebase_storage`
JSON documents / Dart converters	Every Model class has `toMap()` and `fromMap()` to convert to and from Firestore documents.	data layer	plain Dart
Locale data (CLDR via intl)	Turkish and English formatting of amounts and dates (₺1.250,50, 12 Nov 2026).	all	`intl`, Flutter localisations
Dart interfaces (abstract classes)	The contract between ViewModels and data: `ExpenseRepository` has `watchExpenses(gid)`, `addExpense(e)`, `deleteExpense(id)`. The Firebase class implements it; tests use a fake.	S2–S9	plain Dart + `provider`

7 Security, offline behaviour and testing

- **Authorisation:** Firestore Security Rules allow a user to read or write `groups/{gid}` and its sub-collections only if their uid is in `memberIds`; expenses must pass a server-side check that shares sum to the amount.
- **Privacy:** only the minimum personal data is stored (name, e-mail); receipt images are private; sign-out removes the local cache and the FCM token.
- **Offline-first:** writes go to the local Firestore cache first and sync later; the UI shows a "Waiting to sync" chip (see the user flow edge cases).
- **Testing:** unit tests for money rounding and the debt solver (with property tests: balances sum to zero, at most $n-1$ payments), ViewModel tests with fake repositories, and widget tests for S5, S6 and S7.

> **Known limitation.** Server-side push needs a Cloud Function, which requires Firebase's pay-as-you-go plan. If we stay on the free plan for the course, the app will fall back to Firestore listeners and local notifications while it is open, and push will be demonstrated with a test message from the Firebase console.

8 Implementation order (after this milestone)

1. Project setup, theme from the design tokens (`ColorScheme.fromSeed(#0B6B63)`, Plus Jakarta Sans + Inter) and routes for S1–S9.
2. Model classes, `money.dart`, `debt_solver.dart` with unit tests (pure Dart, no Firebase).
3. Repository interfaces with in-memory fakes, then screens S2, S5, S6, S7 running on the fakes.
4. Firebase Auth and Firestore implementations, invite links and QR, then push notifications.